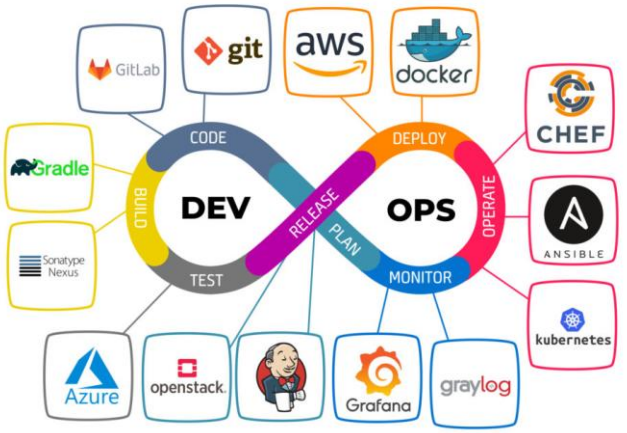
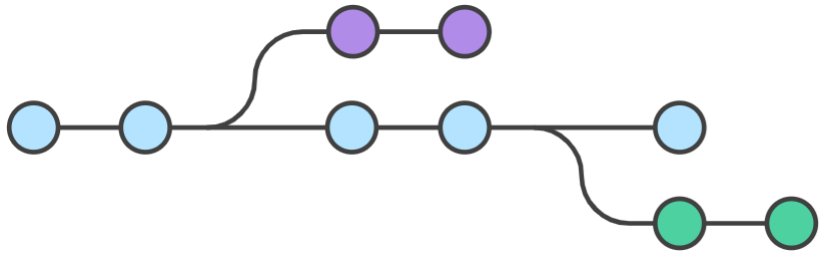
TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie



Git & GitHub

CHAPITRE 5

GIT MERGE CONFLIT



Git Merge conflit

D'où vient les conflits de merge

- Les conflits surviennent généralement lorsque :

  - si deux personnes ont modifié les mêmes lignes dans un fichier,

  - si un développeur a supprimé un fichier alors qu'un autre développeur le modifiait.

- Dans ces cas, Git ne peut pas déterminer automatiquement la version correcte.

- Les **conflits** n'affectent que le développeur qu'effectue le **merge**, les autres membres de l'équipe ne sont pas conscients du conflit.

- Git marquera le fichier comme étant en conflit et arrêtera le processus de merge.

- Il impose alors aux développeurs de résoudre le conflit.

Git Merge conflit

Git ne parvient pas à lancer le merge

Types des conflits de merge

- Un merge peut entrer dans un état de conflit en deux points distincts ;
  - Au début d'un processus de merge
  - Au cours d'un processus de merge
- Le démarrage du merge échouera tant que Git verra qu'il y a des changements dans le répertoire du travail ou la zone d'indexe du projet en cours.
- Git ne parvient pas à lancer le merge, parce que ces changements en suspens pourraient être remplacés par les commits mergés.

➔ Le cas échéant n'est pas dû à des conflits avec d'autres développeurs, mais bien avec des changements locaux en cours.

➔ L'échec d'un merge au démarrage renverra le message d'erreur suivant :

error: Entry '<fileName>' not uptodate. Cannot merge. (Changes in working directory)

Git Merge conflit

Git rencontre un problème durant le merge

Types des conflits de merge

- Un échec PENDANT un merge indique un conflit entre la branche locale actuelle et la branche mergée.
- Cela indique un **conflit avec le code d'autres développeurs**.
- Git fera de son mieux pour merger les fichiers, mais vous laissera le soin de résoudre manuellement les conflits dans les fichiers concernés.
- Un échec en cours de merge renverra le message d'erreur suivant :

error: Entry '<fileName>' would be overwritten by merge. Cannot merge. (Changes in staging area)

Git Merge co

➔ Afin de nous familiari
avec les conflits de merge
produire un conflit à exar
résoudre ultérieurement.

- Créez un répertoire git
basculez vers ce répertoi
initialisez-le git init.

- Créez un fichier
texte merge.txt avec du c

- Ajoutez et commitez n
dépôt.

➔ Nous disposons à pré
nouveau dépôt avec une b
(master ou main) et un
fichier merge.txt avec du

```
user@DESKTOP-RRVCC4I MINGW64 ~
$ mkdir git-merge-test

user@DESKTOP-RRVCC4I MINGW64 ~
$ cd git-merge-test

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test
$ git init
Initialized empty Git repository in C:/Users/user/git-merge-test/.git/

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ echo "contenu à supprimer" > merge.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ cat merge.txt
contenu à supprimer

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git add merge.txt
warning: in the working copy of 'merge.txt', LF will be replaced by CRLF the nex
t time Git touches it

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   merge.txt


user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git commit -am "validation du contenu initial"
[master (root-commit) 17d7357] validation du contenu initial
 1 file changed, 1 insertion(+)
 create mode 100644 merge.txt
```

Git Merge conflit

Création d'un conflit de merge

- Nous allons maintenant créer une branche à utiliser en tant que merge conflictuel.
- Créer une branche nommée new_branch_to_merge_later et en faire un check-out
- Remplacer le contenu dans le fichier merge.txt
- Fai
- Av

remp

```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git checkout -b new_branch_to_merge_later
Switched to a new branch 'new_branch_to_merge_later'

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (new_branch_to_merge_later)
$ echo "contenu totalement différent à fusionner plus tard" > merge.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (new_branch_to_merge_later)
$ git commit -am"modifié le contenu de merge.txt pour provoquer un conflit"
warning: in the working copy of 'merge.txt', LF will be replaced by CRLF the next time Git touches it
[new_branch_to_merge_later b2eae78] modifié le contenu de merge.txt pour provoquer un conflit
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Git Merge conflit

Création d'un conflit de merge

- Cette chaîne de commandes fait un check-out de la branche (master), ajoute du contenu au fichier merge.txt et le commite.

- Notre exemple de dépôt se retrouve ainsi à un état où nous avons deux nouveaux commits. Un dans la branche (master) et un dans la branche new_branch_to_merge_later.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (new_branch_to_merge_later)
$ git checkout master
Switched to branch 'master'

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ echo "contenu à ajouter" >> merge.txt

user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git commit -a -m "contenu ajouté à merge.txt"
warning: in the working copy of 'merge.txt', LF will be replaced by CRLF the next time Git touches it
[master 8fed885] contenu ajouté à merge.txt
 1 file changed, 1 insertion(+)
```

Git Merge conflit

Création d'un conflit de merge

- À présent, exécutons la commande git merge new_branch_to_merge_later pour voir ce qui se produit !

```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git merge new_branch_to_merge_later
Auto-merging merge.txt
CONFLICT (content): Merge conflict in merge.txt
Automatic merge failed; fix conflicts and then commit the result.
```

- Un conflit apparaît.

Git Merge conflit

Comment identifier les conflits de merge

- Comme nous l'avons vu dans l'exemple de la procédure, Git générera une sortie descriptive nous permettant de savoir qu'un CONFLIT s'est produit.
- Pour mieux interpréter, nous pouvons exécuter la commande git status.

```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   merge.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

- La sortie de la commande git status indique que des chemins n'ont pas été mergés en raison d'un conflit.
- Le fichier merge.text apparaît désormais à un état modifié.

Git Merge conflit

Comment identifier les conflits de merge

- Examinons le contenu du fichier en utilisant la commande cat et découvrons ce qui a été modifié.
- Nous pouvons voir certains ajouts étranges.

- <<<<<<< HEAD
- =======
- >>>>>>> new_branch_to_merge_later

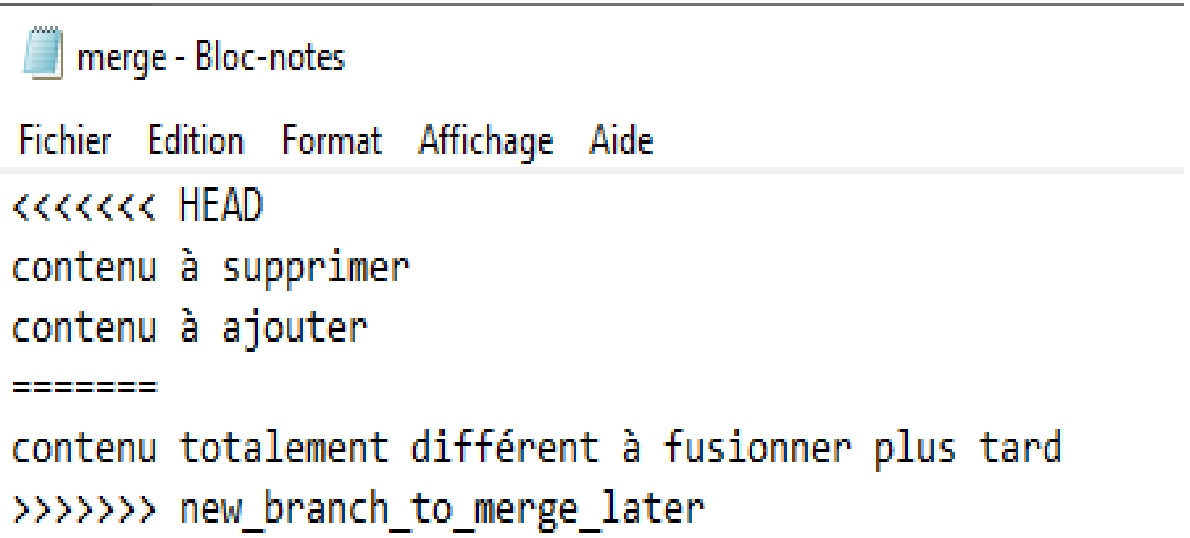
```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master|MERGING)
$ cat merge.txt
<<<<<<< HEAD
contenu à supprimer
contenu à ajouter
=======
contenu totalement différent à fusionner plus tard
>>>>>>> new_branch_to_merge_later
```

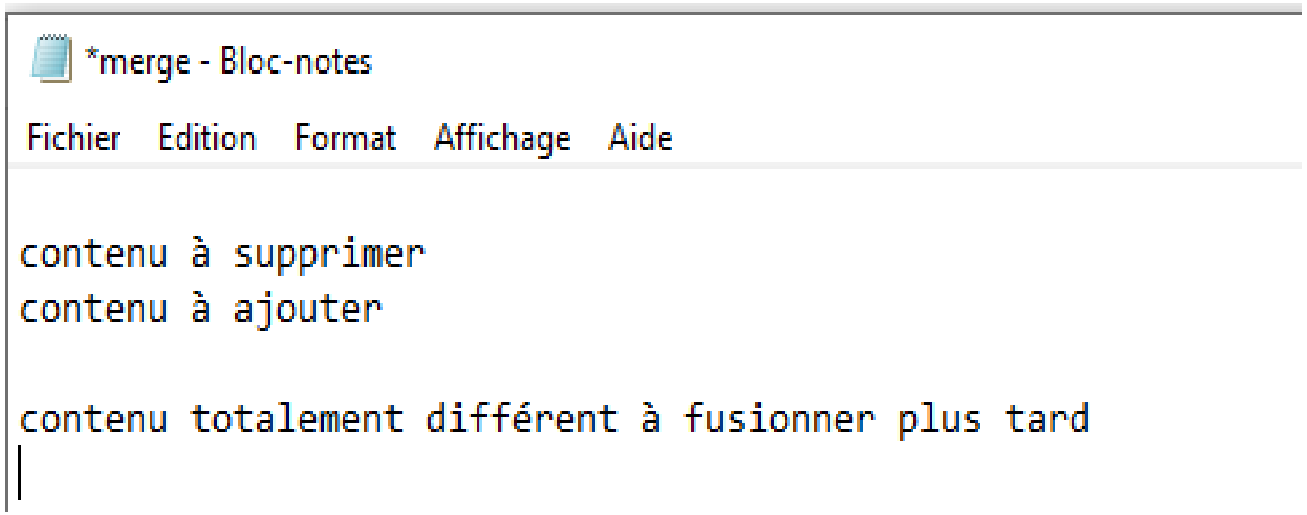
- Considérez ces nouvelles lignes comme des « séparateurs de conflit ».
- La ligne ======= est le « centre » du conflit.
- Tout le contenu entre le centre et la ligne <<<<<<< HEAD est dans la branche principale actuelle vers laquelle pointe la réf HEAD.
- Autrement, tout le contenu entre le centre et >>>>>>> new_branch_to_merge_later est présent dans notre branche de merge.

Git Merge conflit

Comment résoudre les conflits de merge avec la ligne de commande ?

- La manière la plus directe de résoudre un conflit de merge consiste à modifier le fichier conflictuel.
- Ouvrez le fichier merge.txt dans notre éditeur.
- Dans notre exemple, supprimons simplement tous les séparateurs de conflit.
- Le contenu du fichier merge.txt modifié devrait ressembler à ceci :





Git Merge conflit

Comment résoudre les conflits de merge avec la ligne de commande ?

- Une fois le fichier modifié, utilisez git add merge.txt pour indexer le nouveau contenu mergé.
- Pour finaliser le merge, créez un commit :

```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master|MERGING)
$ git commit -a -m "merger et résoudre le conflit de merge.txt"
[master fe87c67] merger et résoudre le conflit de merge.txt
```

- Git voit que le conflit a été résolu et crée un commit de merge pour finaliser le merge.
- Ajoutant la commande git log --oneline --graph --decorate

```
user@DESKTOP-RRVCC4I MINGW64 ~/git-merge-test (master)
$ git log --oneline --graph --decorate
*   fe87c67 (HEAD -> master) merger et résoudre le conflit de merge.txt
|\
| * b2eae78 (new_branch_to_merge_later) modifié le contenu de merge.txt pour provoquer un conflit
* | 8fed885 contenu ajouté à merge.txt
|/
* 17d7357 validation du contenu initial
```

Git Merge conflit

Les commandes Git qui peuvent aider à résoudre des conflits de merge

Outils généraux

$ git status // permet d'identifier les fichiers en conflit durant un merge.

$ git log --merge // --merge génère un journal contenant une liste de commits en conflit entre les branches de merge.

$ git diff // permet de trouver les différences entre les états d'un dépôt/de fichiers. Cette commande est utile pour prévoir et éviter les conflits de merge.

Git Merge conflit

Les commandes Git qui peuvent aider à résoudre des conflits de merge

Outils utiles quand Git ne parvient pas à démarrer un merge

$ git checkout // permet d'annuler les changements apportés à des fichiers ou de modifier des branches.

$ git reset --mixed // reset peut être utilisée pour annuler des changements apportés au répertoire de travail et à la zone d'index.

Git Merge conflit

Les commandes Git qui peuvent aider à résoudre des conflits de merge

Outils utiles en cas de conflits Git durant un merge

$ git merge --abort // l'ajout de --abort mettra fin au processus de merge et réinitialisera la branche à son état antérieur au merge.

$ git reset // reset peut être utilisée durant un conflit de merge pour réinitialiser les fichiers en conflit à un état fonctionnel connu.

Git : Stashing ou cachette

Si un développeur est en train de travailler sur un feature et une tache urgente s'est avérée, il voulait sauvegarder son travail mais sans faire le commit (faute d'organisation). Il est possible de sauvegarder son état actuel grâce à la commande « git stash »

$ git stash // stocke temporairement les changements apportés à la copie de travail pour que nous puissions effectuer d'autres tâches, puis revenir et les réappliquer par la suite

git stash : permet de sauvegarder seulement l'état des fichiers existants dans le staging area

git stash -a : permet de sauvegarder l'état des fichiers existants dans le staging area et l'untracked area (mm un nouveau fichier non encore suivi par le git)

git stash list : pour lister les différentes modifications non commitées

git stash apply id_stash : pour retourner à un état spécifique en fournissant son id (à partir du stash list) :

Exemple : git stash apply stash@{0}

git stash pop id_stash : permet de retourner à un état spécifique en vidant la cache où on a sauvegardé la liste des stashs (modifications)